

TICKETIA

Plataforma de IA para la Gestion Empresarial Automatizada

Analisis Tecnico Completo para Defensa Academica

Trabajo Fin de Master — Ingenieria de Sistemas IA

Alejandro Brata | 2026

Indice de Contenidos

1.	Mapa Completo del Proyecto — Estructura de Carpetas y Archivos	3
2.	Modulos Principales y sus Responsabilidades	6
3.	Sistema de Agentes — Tipos, Funciones y Comunicacion	9
3.1.	Agentes Reactivos	10
3.2.	Agentes Especializados invocados por el Manager	11
3.3.	Agentes Proactivos y Scheduler	12
3.4.	Consejo Estrategico Multi-Persona	13
3.5.	Flujo de Comunicacion Global	14
4.	Librerias del requirements.txt — Rol en el Proyecto	15
5.	Puntos Criticos para el Tribunal	19
5.1.	Decisiones de Arquitectura y su Justificacion	19
5.2.	Debilidades Conocidas y Respuestas Preparadas	22
5.3.	Preguntas Frecuentes de Tribunal — Respuestas Modelo	24

1. Mapa Completo del Proyecto — Estructura de Carpetas y Archivos

Ticketia es una plataforma SaaS multi-tenant de asistencia empresarial basada en Inteligencia Artificial. El código fuente reside en la carpeta **TICKETIA_PRO/** dentro del repositorio, con una organización modular clara que separa responsabilidades por dominios funcionales: rutas web, lógica de negocio, agentes IA, servicios externos y configuración del núcleo.

1.1. Estructura raíz del repositorio

El repositorio contiene, además del código fuente principal, ficheros de infraestructura para despliegue (Docker, docker-compose), documentación académica y scripts auxiliares:

Fichero / Carpeta	Descripción
TICKETIA_PRO/	Directorio principal — toda la lógica de la aplicación
Dockerfile	Imagen Docker para despliegue en contenedor
docker-compose.yml	Orquestación de servicios: app + base de datos
requirements.txt	Dependencias Python del proyecto
ARCHITECTURE_DEFENSE.Doc	Documento de arquitectura para la defensa
PROPUESTA_TFM_TICKETIA.Pdf	Propuesta original del Trabajo Fin de Master
zip_delivery.py	Script para empaquetar la entrega académica
.dockerignore	Exclusiones del contexto Docker
test_docker/	Tests de validación del contenedor

1.2. Estructura interna de TICKETIA_PRO/

La aplicación sigue una arquitectura en capas con separación clara entre presentación (routes, templates), lógica de negocio (modules) y configuración (core):

Ruta	Tipo	Descripción
app.py	.py	Entry point: inicializa Flask, BD, admin panel, blueprints
mcp_server.py	.py	Servidor MCP — expone herramientas al LLM vía protocolo MCP/stdio
mcp_server_sse.py	.py	Variante SSE del servidor MCP para conexiones de larga duración
core/config.py	.py	Gestión de variables de entorno y configuración global
core/db_models.py	.py	Modelos SQLAlchemy — toda la estructura de la base de datos

core/clients.py	.py	Clientes singleton: OpenAI, Twilio (evitan reconexiones)
core/mcp_client.py	.py	Cliente async que lanza mcp_server como subprocess y ejecuta tools
core/notifier.py	.py	Sistema de notificaciones internas en la plataforma web
core/storage.py	.py	Gestion de ficheros subidos (tickets, imagenes, audios)
modules/agents/manager.py	.py	Motor principal: recibe mensaje, llama GPT-4o, ejecuta tools
modules/agents/tools.py	.py	Definicion JSON de herramientas para el LLM (function calling)
modules/agents/history.py	.py	Memoria conversacional — recupera ultimas N interacciones
modules/agents/background_tasks.py	.py	Procesamiento async en hilos secundarios (marketing)
modules/chatbot/logic.py	.py	Chatbot simple de fallback sin herramientas
modules/council/orchestrator.py	.py	Debate multi-persona: 3 roles IA en 3 rondas + sintesis
modules/proactive/scheduler.py	.py	Planificador de agentes proactivos con libreria schedule
modules/proactive/admin_redactor.py	.py	Agente: clasificacion de imagenes y redaccion de docs
modules/proactive/business_health.py	.py	Agente: analisis de salud financiera del negocio
modules/proactive/grant_hunter.py	.py	Agente: busqueda de subvenciones en BOE y web
modules/proactive/marketing_agent.py	.py	Agente: generacion de contenido de marketing (img/ppt/video)
modules/proactive/networker.py	.py	Agente: deteccion de sinergias entre usuarios
modules/proactive/post_sales.py	.py	Agente: gestion de postventa y atencion al cliente
modules/services/document.py	.py	Generacion de PDFs con fpdf y presentaciones con python-pptx
modules/services/notification.py	.py	Envio de emails (Flask-Mail) y mensajes WhatsApp (Twilio)
modules/services/whatsapp_dispatcher.py	.py	Dispatcher: clasifica y enruta mensajes WhatsApp entrantes
modules/tickets/logic.py	.py	OCR y procesamiento de tickets/facturas via vision GPT-4o
modules/utils/transcriber.py	.py	Transcripcion de audio con Whisper API de OpenAI
routes/api.py	.py	Endpoints REST: chat, upload ticket, upload audio, council stream
routes/web.py	.py	Rutas HTML: autenticacion, dashboard, documentos, wizard
routes/webhooks.py	.py	Webhook Twilio: recepcion de mensajes WhatsApp
llmops/eval_agents.py	.py	Evaluacion de calidad de respuestas con DeepEval (LLMOps)
tests/test_proactive_agents.py	.py	Tests de agentes proactivos con pytest
instance/zeptai.db	.db	Base de datos SQLite para desarrollo local

1.3. Carpeta templates/ — Vistas HTML

Las plantillas Jinja2 siguen una estructura jerarquica con herencia: **base.html** define el layout global (navbar, sidebar, notificaciones) y el resto extiende este template base.

Template	Descripcion
base.html	Layout maestro: navbar, sidebar, barra de notificaciones
landing.html	Pagina de inicio publica con presentacion del producto
login.html / register.html	Autenticacion: inicio de sesion y registro de nuevos usuarios
forgot_password.html	Formulario de recuperacion de contrasena por email
dashboard.html	Panel principal: metricas, graficas, actividad reciente
transactions.html	Listado de tickets/facturas con filtros y acciones
documents.html	Galeria de documentos generados agrupados por tipo y fecha
agents.html	Marketplace de agentes disponibles con toggle de activacion
agent_config.html	Configuracion individual de cada agente
wizard.html	Onboarding: configuracion inicial del negocio paso a paso
demo_panel.html	Panel de demostracion con datos de ejemplo
council.html	Interfaz del Consejo Estrategico con streaming SSE
components/agent_card.html	Componente reutilizable para tarjetas de agentes

2. Módulos Principales y sus Responsabilidades

El proyecto organiza su logica en tres grandes capas: el nucleo (core), los modulos funcionales (modules) y las rutas (routes). Cada capa tiene una responsabilidad clara y comunica con las adyacentes a traves de interfaces definidas.

2.1. Capa Core — Configuración y Persistencia

app.py — Entry Point y Orquestador de Arranque

Este fichero es el punto de entrada de la aplicacion Flask. Sus responsabilidades son:

- Carga de variables de entorno desde .env mediante python-dotenv
- Inicializacion de la base de datos con Flask-SQLAlchemy y creacion de tablas
- Registro de Blueprints: web, api y webhooks
- Configuracion del panel de administracion con Flask-Admin y vistas protegidas por rol (SecureModelView)
- Context processor global que inyecta el conteo de notificaciones no leidas en todas las vistas
- Endpoints de notificaciones: GET /api/notifications, POST mark_read, POST mark_all_read
- Arranque del servidor en el puerto configurado por variable de entorno PORT (por defecto 5000)

core/db_models.py — Modelos de Base de Datos

Define la estructura completa de datos usando SQLAlchemy ORM. Los modelos principales son:

Modelo	Tabla	Campos clave	Relaciones
BusinessProfile	business_profiles	user_phone, business_name, system_created_timestamp, client_email	One-to-many con Ticket, Appointment, GeneratedDocument, ChatMessage, AgentConfig, Notification, Grant
Ticket	tickets	amount, vendor, category, date, image_path, processed	Many-to-one con BusinessProfile
Appointment	appointments	date, time, client_name, client_phone, name_to_phone	Many-to-one con BusinessProfile
GeneratedDocument	generated_documents	doc_type, file_path, created_at, content	Many-to-many con BusinessProfile
ChatMessage	chat_messages	role, content, timestamp, channel	Many-to-one con BusinessProfile
AgentConfig	agent_configs	agent_id, is_active, config_json	Many-to-one con BusinessProfile
Notification	notifications	message, is_read, type, created_at	Many-to-one con BusinessProfile
Grant	grants	title, description, url, deadline, sector	Independiente
SynergyMatch	synergy_matches	user_a, user_b, match_reason, score	Referencia dos BusinessProfiles

core/clients.py — Clientes Singleton

Implementa el patron Singleton para los clientes de APIs externas. El objetivo es evitar reconexiones innecesarias y centralizar la configuracion de credenciales. Incluye el cliente **OpenAI** (para GPT-4o y Whisper) y el cliente **Twilio** (para WhatsApp). Se inicializan una sola vez al arrancar la aplicacion y se reutilizan en todos los modulos que los necesitan.

2.2. Capa Routes — Presentacion y API

routes/web.py — Rutas HTML (784 lineas)

Es el fichero mas extenso de rutas. Implementa el ciclo completo de vida del usuario en la plataforma:

Ruta	Metodo	Descripcion
/	GET	Landing page publica; redirige al dashboard si hay sesion activa
/register	GET/POST	Registro con validacion de email unico y formato de telefono
/login	GET/POST	Autenticacion por email+password con hashing bcrypt
/logout	GET	Limpia la sesion Flask y redirige al login
/forgot_password	POST	Genera token de reset y envia email con enlace temporal
/reset_password/<token>	GET/POST	Valida token y actualiza contrasena
/dashboard	GET	Metricas del mes: gastos, tickets, chats, actividad reciente
/transactions	GET	Listado completo de tickets con filtros por fecha/categoria
/documents	GET	Galeria de documentos agrupados por tipo (propuestas, facturas, marketing)
/export_excel	GET	Exporta gastos a Excel con formato profesional (XlsxWriter)
/marketplace	GET	Catalogo de agentes disponibles con estado activo/inactivo
/toggle_agent/<id>	POST	Activa o desactiva un agente para el usuario
/agent_config/<id>	GET	Pagina de configuracion detallada del agente
/save_agent_config/<id>	POST	Guarda configuracion del agente en JSON dentro de la BD
/wizard	GET/POST	Wizard de onboarding para configurar el perfil de negocio
/council	GET	Interfaz del Consejo Estrategico
/demo/seed_data	POST	Inyecta datos historicos de ejemplo para demostraciones
/demo/trigger/<agent>	POST	Dispara manualmente un agente proactivo especifico

routes/api.py — Endpoints REST

Endpoint	Descripcion
POST /api/chat	Chat web autenticado; ejecuta AgentExecutor con canal 'web'

POST /upload_web_ticket	Sube imagen de ticket; procesa OCR y extrae datos
POST /upload_web_audio	Sube audio WebM; transcribe con Whisper y ejecuta agente
POST /generate_video_from_image	Genera video con Runway ML desde imagen subida
POST /api/council/stream	Stream SSE del debate del Consejo Estrategico

routes/webhooks.py — Integracion Twilio

Recibe las peticiones HTTP de Twilio cuando llega un mensaje a los numeros WhatsApp configurados. Valida la firma de Twilio (X-Twilio-Signature) para evitar peticiones fraudulentas. Delega el procesamiento al **WhatsAppWebhookDispatcher** en `modules/services/whatsapp_dispatcher.py`.

3. Sistema de Agentes — Tipos, Funciones y Comunicacion

Ticketia implementa una arquitectura multi-agente donde diferentes agentes IA especializados colaboran para dar servicio al usuario. Los agentes se clasifican en tres categorias segun su patron de activacion: **reactivos** (responden al usuario en tiempo real), **especializados** (invocados por el agente principal como subagentes) y **proactivos** (se ejecutan autonomamente en background sin intervencion del usuario).

Patron arquitectonico: El sistema sigue un patron de orquestacion con un agente central (AgentExecutor) que actua como router/orchestrator, delegando en agentes especializados segun el tipo de tarea detectada por el LLM.

3.1. Agentes Reactivos

AgentExecutor — modules/agents/manager.py

Es el cerebro del sistema. Se instancia para cada mensaje entrante y gestiona el ciclo completo de procesamiento. Su flujo de ejecucion es el siguiente:

- **Paso 1:** Recibe: mensaje de texto, numero de telefono, perfil de negocio, URL de media y canal (whatsapp/web)
- **Paso 2:** Si hay imagen adjunta: dispatch directo a AdminRedactor para clasificacion
- **Paso 3:** Guarda la interaccion del usuario en el historial de chat (tabla chat_messages)
- **Paso 4:** Construye el contexto: system prompt del negocio + ultimas 10 interacciones
- **Paso 5:** Primera llamada a GPT-4o con lista de herramientas disponibles (function calling)
- **Paso 6:** Si el modelo devuelve tool_calls: ejecuta cada herramienta secuencialmente
- **Paso 7:** Anade los resultados de herramientas al historial de mensajes
- **Paso 8:** Segunda llamada a GPT-4o para sintetizar resultados de herramientas en respuesta natural
- **Paso 9:** Guarda la respuesta final en el historial y en el log de actividad
- **Paso 10:** Retorna la respuesta final al canal de origen (WhatsApp o web)

Herramientas disponibles para el LLM (definidas en tools.py):

Herramienta	Descripcion	Agente invocado
check_availability	Consulta disponibilidad de agenda para una fecha/hora	Directo en BD
book_appointment	Crea una cita en la base de datos	Directo en BD
create_proposal_from_last_image	Genera PDF de propuesta desde la ultima imagen recibida	AdminRedactor

create_proposal_from_text	Genera PDF desde datos estructurados en texto	AdminRedactor
generate_marketing_material	Lanza generacion async de contenido de marketing	MarketingAgent (background)
handle_customer_service	Deriva al agente de postventa	PostSalesAgent

Chatbot Simple — `modules/chatbot/logic.py`

Agente de fallback que se usa cuando no existe un perfil de negocio configurado. No tiene acceso a herramientas. Simplemente llama al LLM con un system prompt generico e implementa un guardrail para mantener las respuestas en el dominio empresarial. Su funcion principal es orientar a nuevos usuarios hacia el proceso de registro y configuracion.

3.2. Agentes Especializados (Subagentes)

AdminRedactor — `modules/proactive/admin_redactor.py`

Agente dual con dos capacidades principales:

- **Clasificador de imagenes:** Cuando recibe una imagen, usa vision de GPT-4o para determinar si es un borrador/esquema de propuesta o un ticket/factura comercial. Esta clasificacion determina el flujo posterior.
- **Redactor de documentos:** Genera propuestas y presupuestos profesionales en PDF. Puede trabajar desde una imagen (extrae la informacion visualmente) o desde datos estructurados en texto. El PDF se genera con fpdf y se envia por email.

MarketingAgent — `modules/proactive/marketing_agent.py`

Agente de generacion de contenido de marketing. Siempre se ejecuta en un hilo secundario (background_tasks.py) para no bloquear la respuesta HTTP. Puede generar tres tipos de contenido:

- **Imagenes de marketing:** Usa DALL-E 3 o la API de OpenAI para generar imagenes profesionales basadas en el prompt del usuario
- **Presentaciones PowerPoint:** Genera ficheros .pptx con python-pptx con slides estructuradas, titulos, contenido y estilo de marca
- **Videos:** Usa el SDK de Runway ML para generar videos cortos a partir de imagenes de referencia

Una vez generado el contenido, notifica al usuario via WhatsApp (si el canal es WhatsApp) o crea una notificacion interna en la plataforma web. El fichero se guarda en la BD como GeneratedDocument.

PostSalesAgent — `modules/proactive/post_sales.py`

Agente especializado en la gestion de la relacion postventa con clientes. Se activa cuando el LLM detecta intencion de atencion al cliente en el mensaje. Sus capacidades incluyen: gestion de quejas y reclamaciones, seguimiento de pedidos, envio de encuestas de satisfaccion y recordatorios de citas pendientes. Mantiene el tono y la personalidad configurada en el perfil de negocio del usuario.

3.3. Agentes Proactivos y Scheduler

Los agentes proactivos son la característica diferencial de Ticketia respecto a un chatbot convencional. Se ejecutan periódicamente en background, analizan la situación de cada negocio y generan alertas, recomendaciones o acciones sin que el usuario lo solicite.

Agente	Archivo	Frecuencia	Funcion Principal	Output
BusinessHealthAgent	business_health.py	Diaria	Analiza metricas financieras: gastos de ventas, marketing, etc.	Alertas WhatsApp, correos, etc.
GrantHunter	grant_hunter.py	Semanal	Busca subvenciones en BOE, CDTI y webs de ayudas.	Boletín de noticias, alertas.
Networker	networker.py	Semanal	Compara perfiles de negocio en la plataforma de LinkedIn.	Alertas de oportunidades.
MarketingAgent	marketing_agent.py	Bajo demanda	Genera contenido de marketing (imagenes, posts, etc.)	Contenido para redes sociales.

Scheduler — modules/proactive/scheduler.py

Usa la libreria **schedule** de Python para planificar la ejecucion periodica de los agentes. Se ejecuta en un hilo separado al arrancar la aplicacion. Itera sobre todos los BusinessProfiles activos y ejecuta cada agente con el contexto del negocio correspondiente. **Limitacion conocida:** schedule no persiste entre reinicios del servidor; si la aplicacion se reinicia, el scheduler vuelve a empezar desde cero.

3.4. Consejo Estrategico Multi-Persona

CouncilManager — modules/council/orchestrator.py

El Consejo Estrategico es la característica mas innovadora del sistema desde el punto de vista de diseno de agentes. Implementa un debate estructurado entre tres personas IA con perspectivas distintas sobre el mismo LLM (GPT-4o), cada una con un system prompt diferente que define su rol, personalidad y area de conocimiento.

Persona	Icono	Perfil	Area	Estilo de Comunicacion
El Socio	Tigre	Emprendedor agresivo orientado a ventas.	Ventas, expansion, captacion de clientes.	Frases directas, ambicioso
El Gestor	Buho	Experto legal y fiscal conservador.	Complimiento, riesgos, obligaciones.	Precauciones, detallista
El Coach	Cohete	Consultor de productividad y procesos.	Procesos, equipo, bienestar empresarial.	Empujones, practico, motivador

Protocolo de debate — 3 rondas:

- **Ronda 1 — Posiciones iniciales:** Cada persona da su opinion inicial sobre el tema propuesto. Se ejecutan secuencialmente para crear la sensacion de una conversacion natural.
- **Ronda 2 — Replicas:** Cada persona reacciona a las opiniones de las otras dos (maximo 25 palabras por replica). Esta limitacion fuerza argumentos concisos y centrados.
- **Ronda 3 — Sintesis y plan de accion:** Se genera un plan de accion en formato Markdown que integra las tres perspectivas, identificando puntos de consenso y acciones concretas ordenadas por prioridad.

Implementacion tecnica: Usa **AsyncOpenAI** para las llamadas al LLM y un generador de eventos para el streaming via SSE (Server-Sent Events). El frontend consume el stream con EventSource de JavaScript y va mostrando las intervenciones de cada persona en tiempo real. Opcionalmente, cada persona puede usar herramientas MCP durante sus intervenciones para consultar datos reales (e.g., El Gestor puede buscar normativa fiscal vigente con search_web).

3.5. Flujo de Comunicacion Global

El siguiente diagrama describe el flujo completo desde que un mensaje entra al sistema hasta que se genera la respuesta:

```
ENTRADA WhatsApp
  ■■> webhooks.py (valida firma Twilio)
    ■■> WhatsAppDispatcher
      ■■ ¿Numero dedicado del cliente?
      ■ ■■> AgentExecutor (negocio especifico)
      ■■ ¿Numero central de Ticketia?
        ■■ ¿Audio? ■■> Whisper (transcripcion) ■■> AgentExecutor
        ■■ ¿Imagen? ■■> AdminRedactor (clasificar)
        ■ ■■ Borrador ■■> AgentExecutor (con media_url)
        ■ ■■ Ticket ■■■> tickets/logic.py (OCR)
        ■■ ¿Texto? ■■> AgentExecutor

DENTRO de AgentExecutor:
  ■■> GPT-4o (con tools + historial + system prompt)
    ■■ tool: book_appointment ■■> BD directa
    ■■ tool: create_proposal ■■> AdminRedactor ■■> PDF ■■> Email
    ■■ tool: generate_marketing ■■> background_tasks ■■> MarketingAgent
    ■■ tool: handle_cs ■■> PostSalesAgent
    ■■ tool: [MCP tools] ■■> mcp_client ■■> mcp_server
      ■■ get_financial_summary (BD)
      ■■ search_web (DuckDuckGo)
      ■■ schedule_appointment (BD)
      ■■ send_email_notification (SMTP)

ENTRADA Web (chat en dashboard)
  ■■> routes/api.py POST /api/chat
    ■■> AgentExecutor (canal='web')
      [mismo flujo que arriba]

COUNCIL (streaming):
  ■■> routes/api.py POST /api/council/stream
    ■■> CouncilManager.run_session()
      ■■ Ronda 1: El Socio, El Gestor, El Coach (opiniones)
      ■■ Ronda 2: replicas cruzadas
      ■■ Ronda 3: sintesis ■■> SSE stream ■■> frontend

PROACTIVO (background, sin usuario):
  ■■> scheduler.py (hilo separado)
    ■■ BusinessHealthAgent ■■> analisis + notificacion
    ■■ Granthunter ■■> busqueda web + guardado en BD
```

■ ■ Networker ■ ■ > comparacion perfiles + SynergyMatch

4. Librerías del requirements.txt — Rol en el Proyecto

A continuacion se detalla cada dependencia del proyecto con su uso concreto, el modulo que la consume y la justificacion de su eleccion frente a alternativas.

4.1. Framework Web y Base de Datos

Libreria	Version	Modulo que la usa	Uso concreto en Ticketia	Alternativas consideradas
flask	>=2.x	app.py, routes/*	Framework web principal: routing, sesiones, API (reprints, tickets, etc)	FastAPI (reprints, tickets, etc) o Django (reprints, tickets, etc)
flask-sqlalchemy	>=3.x	core/db_models.py, todos los modelos	ORM que mapea clases Python a tablas SQL	SQLAlchemy (reprints, tickets, etc) o Peewee (reprints, tickets, etc)
flask-mail	>=0.9	modules/services/notificaciones.py, app.py	Envío de emails transaccionales: documentos PDF (ajustes de logs), etc	smtpd (reprints, tickets, etc) o SendGrid (reprints, tickets, etc)
flask-admin	>=1.6	app.py	Panel de administracion con CRUD automático de modelos	Admin-on-models (reprints, tickets, etc) o Flask-Admin (reprints, tickets, etc)
psycopg2-binary	>=2.9	core/config.py (DB URL)	Driver de conexion PostgreSQL para asyncpg	asyncpg (reprints, tickets, etc) o psycopg2 (reprints, tickets, etc)

4.2. Inteligencia Artificial y LLM

Libreria	Modulo que la usa	Uso concreto en Ticketia
openai	manager.py, council/orchestrator.py, ticket_parser.py, nlp_helpers.py	Modelo de lenguaje para generación de texto; GPT-4o Vision para OCR de tickets
mcp[cli]	mcp_server.py, mcp_server_sse.py	Implementación de MCP Model Context Protocol de Anthropic. Se usa FastMCP para definir herramientas
duckduckgo-search	mcp_server.py, proactive/grant_hunter.py	Motor de búsqueda web privado y gratuito. Se usa en la herramienta search_web del MCP
deepeval	llmops/eval_agents.py	Framework de evaluacion de calidad de respuestas LLM. Mide metricas como Answer Relevance
runwayml	proactive/marketing_agent.py	SDK oficial de Runway ML para generacion de videos con IA. Se usa para crear videos de marketing

4.3. Integraciones Externas

Libreria	Modulo que la usa	Uso concreto en Ticketia
twilio	routes/webhooks.py, modules/services/integrations.py	Integración con la API de WhatsApp Business via Twilio. Recibe webhooks de mensajes
requests	proactive/grant_hunter.py, proactive/marketing_agent.py	Comunicación con APIs externas: descarga de archivos multimedia de Twilio, llamadas a APIs
python-dotenv	core/config.py	Carga de variables de entorno desde el fichero .env: API keys de OpenAI y Twilio, credenciales de Twilio

4.4. Generacion de Documentos

Libreria	Modulo que la usa	Uso concreto en Ticketia
----------	-------------------	--------------------------

fpdf	modules/services/document.py, proactive/admin.py	Generación de PDFs programaticos: propuestas comerciales, presupuestos, facturas p
python-pptx	proactive/marketing_agent.py	Generacion de presentaciones PowerPoint con diapositivas estructuradas. El Marketin
Pillow	proactive/marketing_agent.py, proactive/admin.py	Procesamiento de imágenes: redimensionado antes de enviar a la API de vision, conv
matplotlib	proactive/business_health.py	Generacion de graficas de metricas financieras (gastos por categoria, tendencia mens
xlsxwriter	routes/web.py (/export_excel)	Generacion de ficheros Excel con formato profesional: colores alternados por fila, form

4.5. Utilidades y Procesamiento de Datos

Libreria	Modulo que la usa	Uso concreto en Ticketia
pandas	routes/web.py, proactive/business_health.py	Manipulacion de datos tabulares para calculo de metricas: agrupacion de gastos por c
python-dateutil	modules/tickets/logic.py, routes/web.py	Parsing flexible de fechas extraidas de tickets por OCR. Las facturas vienen con forma
schedule	modules/proactive/scheduler.py	Planificacion de tareas periodicas con sintaxis declarativa (schedule.every().day.do(fn)
gunicorn	Despliegue (Dockerfile, docker-compose)	Servidor WSGI de produccion. Reemplaza al servidor de desarrollo de Flask. Gestiona
pytest	tests/	Framework de testing. Se usa para los tests unitarios de los agentes proactivos y los t

5. Puntos Criticos para el Tribunal

Esta seccion prepara al autor para responder a las preguntas mas exigentes de un tribunal academico. Cada decision tecnica importante se analiza desde tres angulos: que se hizo, por que se eligio esa opcion y cuales son sus limitaciones conocidas.

5.1. Decisiones de Arquitectura y su Justificacion

Decision 1: Flask sobre FastAPI

Que se hizo: Se eligio Flask como framework web principal en lugar de FastAPI.

Por que: El 95% de los endpoints del proyecto son sincronos (request/response clasico). Flask tiene un ecosistema mas maduro para las integraciones necesarias: Flask-SQLAlchemy, Flask-Mail y Flask-Admin estan extremadamente bien integrados y permiten construir el panel de administracion, las vistas web y la gestion de BD con mucho menos codigo. FastAPI anade complejidad async que solo seria beneficosa si todos los endpoints fueran async, lo que requeriria un ORM async (SQLAlchemy async) y complicaria el codigo sin beneficio proporcional en un MVP.

Atencion: El Consejo Estrategico usa asyncio dentro de una app Flask sincrona, lo que requiere `asyncio.run()` o un loop de eventos separado. Esto es una inconsistencia arquitectonica menor que en una version production-ready se resolveria migrando el council a un servicio independiente con FastAPI.

Decision 2: Model Context Protocol (MCP)

Que se hizo: Se implemento un servidor MCP (`mcp_server.py`) usando FastMCP, y un cliente async (`mcp_client.py`) que lo consume via subprocess con protocolo stdio.

Por que: MCP es el estandar emergente definido por Anthropic para que los LLMs accedan a herramientas y contexto externo de forma estandarizada. Su adopcion en el proyecto tiene valor academico y tecnico: desacopla la definicion de herramientas del codigo del agente, permite que el LLM descubra herramientas en runtime (lista de tools dinamica), y prepara la arquitectura para integracion con cualquier cliente MCP en el futuro (Claude Desktop, otros modelos que soporten MCP).

Herramientas expuestas via MCP:

- `get_financial_summary(user_phone)`: consulta la BD y devuelve resumen de gastos del usuario
- `search_web(query, max_results)`: busca informacion actual con DuckDuckGo (subvenciones, noticias)
- `schedule_appointment(owner, date, time, client)`: crea citas comprobando conflictos en BD
- `send_email_notification(to, subject, body)`: envia emails via SMTP/Flask-Mail

Atencion: El servidor MCP se lanza como subprocess con stdio, lo que anade latencia de arranque (fork de proceso) y complejidad de gestion. Para entornos de alta concurrencia, la

variante SSE (mcp_server_sse.py) es mas apropiada al mantener conexiones persistentes.

Decision 3: Council — Role Prompting vs. Multi-Agente Real

Que se hizo: El Consejo Estrategico implementa tres 'personas' sobre el mismo modelo GPT-4o, cada una con un system prompt diferente que define su rol, personalidad y area de expertise.

Por que: El diseno de personas (role prompting) sobre un unico LLM es un patron bien documentado en la literatura de sistemas multi-agente basados en LLMs. Ofrece varias ventajas: consistencia del modelo base (el mismo GPT-4o para todas las perspectivas), menor costo (un proveedor, una API), menor latencia (sin overhead de comunicacion entre modelos distintos), y suficiente diversidad cognitiva para los propositos del sistema (asesoramiento empresarial).

Pregunta critica esperada: 'No es esto un unico agente con diferentes prompts? Como es multi-agente?' **Respuesta:** Correcto, es un unico modelo con role prompting, lo que en la literatura se denomina 'persona-based multi-agent simulation'. La distincion es honesta: no hay modelos separados ni comunicacion real entre agentes. El valor esta en el protocolo de debate (3 rondas estructuradas) y la sintesis integrada, no en la heterogeneidad de modelos. Para una version v2 se podrian usar modelos especializados (GPT-4o para El Socio, Claude para El Gestor, Gemini para El Coach).

Decision 4: WhatsApp como canal principal

Que se hizo: Se eligio WhatsApp via Twilio como canal de comunicacion principal en lugar de desarrollar una app movil nativa o una interfaz web-first.

Por que: El publico objetivo son autonomos y PYMEs espanolas. Segun datos de Statista (2024), el 93% de los espanoles usa WhatsApp diariamente. La adopcion es cero: no hay que instalar nada ni cambiar habitos. El usuario puede enviar un ticket fotografiandolo directamente desde WhatsApp, recibir sus documentos por el mismo canal y interactuar con el asistente con el mismo flujo que usa para hablar con clientes o proveedores.

Consideracion tecnica: Twilio actua como capa de abstraccion sobre la API de WhatsApp Business. Gestiona el numero de telefono, la validacion de mensajes (firma HMAC en cabecera X-Twilio-Signature) y el envio de archivos multimedia. La alternativa directa (API oficial de WhatsApp Business de Meta) requiere proceso de verificacion empresarial que no es viable para un MVP academico.

Decision 5: Multi-tenancy en base de datos compartida

Que se hizo: Todos los negocios (BusinessProfiles) comparten la misma base de datos PostgreSQL. El aislamiento se implementa a nivel de aplicacion filtrando por user_phone en todas las consultas.

Por que: Es el patron multi-tenant mas simple (shared schema, shared database). Simplifica enormemente el deployment (una sola instancia de PostgreSQL), las migraciones de esquema (se aplican a todos los tenants a la vez) y el mantenimiento. Para un MVP con un numero limitado de usuarios, el rendimiento es suficiente.

Atencion: El modelo actual no tiene Row Level Security (RLS) en la base de datos. El aislamiento depende exclusivamente del codigo de aplicacion. Un bug en el filtrado podria exponer datos de un negocio a otro. En produccion real se implementaria RLS en PostgreSQL o un esquema por tenant.

Decision 6: GPT-4o sobre modelos open-source

Que se hizo: Se usa exclusivamente GPT-4o de OpenAI como modelo de lenguaje.

Por que: Las tareas mas criticas del sistema requieren alta precision en espanol: OCR de tickets y facturas (con formatos variados, escritura manual, sellos), extraccion estructurada de datos (importe, proveedor, fecha, categoria fiscal), y generacion de documentos legales/comerciales. Los benchmarks propios y de la literatura muestran que GPT-4o supera a los modelos open-source disponibles en 2024-2025 en estas tareas especificas en espanol.

Riesgos conocidos: Dependencia de un proveedor unico (vendor lock-in), costo variable en funcion del numero de tokens, latencia de red hacia servidores de OpenAI, y cambios de precio o disponibilidad fuera del control del proyecto.

5.2. Debilidades Conocidas y Respuestas Preparadas

Debilidad: Autenticacion basica (sesiones Flask sin JWT)

Descripcion del problema: El sistema usa sesiones del lado del servidor con Flask-Session. No hay JWT, no hay OAuth2, no hay 2FA.

Respuesta preparada: Es suficiente para un MVP con usuarios registrados. En produccion se implementaria OAuth2 (Google/Microsoft) para el segmento empresarial y JWT para los endpoints de la API REST. La contraseña se almacena con hashing bcrypt, lo que es correcto.

Debilidad: Scheduler sin persistencia entre reinicios

Descripcion del problema: La libreria schedule ejecuta tareas en memoria. Si el servidor se reinicia, las tareas pendientes se pierden y el scheduler empieza desde cero.

Respuesta preparada: Limitacion conocida y documentada del MVP. En produccion se usaria Celery + Redis (broker de tareas persistente) o APScheduler con backend de base de datos. La decision de usar schedule priorizo la simplicidad para el prototipo academico.

Debilidad: Sin rate limiting en la API

Descripcion del problema: Los endpoints de la API REST y el webhook de WhatsApp no tienen limitacion de frecuencia, lo que los hace vulnerables a abusos.

Respuesta preparada: El webhook de Twilio esta protegido por validacion de firma HMAC, lo que previene el abuso desde fuentes externas. Para los endpoints de la API web se usaria Flask-Limiter en produccion. En el contexto academico del MVP, el riesgo es asumible.

Debilidad: MCP via subprocess (latencia y complejidad)

Descripcion del problema: El servidor MCP se lanza como un nuevo proceso Python cada vez que el cliente MCP necesita ejecutar herramientas, anadiendo latencia de fork.

Respuesta preparada: La arquitectura MCP esta disenada para ser modular: el servidor se puede externalizar a un proceso persistente o usar la variante SSE (mcp_server_sse.py). El valor de usar MCP es la interoperabilidad estandarizada, no el rendimiento optimo en el primer prototipo.

Debilidad: Tests limitados (cobertura parcial)

Descripcion del problema: El proyecto tiene tests de agentes proactivos y tests de sanidad, pero la cobertura de codigo es baja en modulos criticos como el AgentExecutor.

Respuesta preparada: El sistema incluye evaluacion LLMops con DeepEval (llmops/eval_agents.py), que es la forma correcta de evaluar sistemas IA: medir calidad de respuestas, relevancia contextual y fidelidad. Los tests unitarios clasicos son menos significativos para sistemas donde el comportamiento depende del LLM.

Debilidad: API keys en fichero .env

Descripcion del problema: Las credenciales de OpenAI, Twilio y la BD se gestionan via variables de entorno en un fichero .env que no se sube al repositorio.

Respuesta preparada: Es el estandar de la industria para desarrollo. En produccion se usaria un secrets manager: AWS Secrets Manager, HashiCorp Vault o el sistema de secrets de Kubernetes. El .env esta en .gitignore y nunca se ha subido al repositorio.

5.3. Preguntas Frecuentes de Tribunal — Respuestas Modelo

P: Como garantizas que los datos de un cliente no sean visibles por otro?

R: El aislamiento multi-tenant se implementa a dos niveles: (1) Nivel de aplicacion: todas las consultas SQLAlchemy filtran por user_phone, que actua como identificador de tenant. (2) Nivel de sesion: el user_phone del usuario autenticado se almacena en la sesion Flask y se usa para filtrar cada consulta sin posibilidad de que el usuario lo manipule. La mejora para produccion seria Row Level Security en PostgreSQL.

P: Que pasa si OpenAI tiene una interrupcion de servicio?

R: El sistema queda degradado: los agentes IA no pueden responder. No hay fallback a otro LLM en la version actual, lo que es un single point of failure. La mitigacion a corto plazo seria implementar retry con exponential backoff (ya parcialmente presente en las llamadas a la API). La mitigacion a largo plazo seria un router de LLMs que cambie a Anthropic Claude o Gemini si OpenAI no responde en un tiempo maximo.

P: Como evaluas la calidad de las respuestas de los agentes?

R: El proyecto incluye un pipeline LLMops en llmops/eval_agents.py usando DeepEval. Se miden tres metricas principales: Answer Relevancy (la respuesta responde a lo que el usuario pregunto), Faithfulness (la respuesta no incluye informacion inventada), y Contextual Precision (se usa el contexto correcto del negocio). Los tests de evaluacion se pueden ejecutar contra casos de uso reales documentados en el gold standard del dataset.

P: Por que no usaste LangChain o LlamaIndex?

R: Decision consciente. LangChain anade una capa de abstraccion que complica el debugging sin aportar valor proporcional cuando el numero de tipos de agente es bajo y controlado. El AgentExecutor del proyecto hace exactamente lo que se necesita: history, tool calling, second call. Escribirlo directamente con la API de OpenAI da control total, menos dependencias y codigo mas mantenible. MCP ya estandariza la parte de herramientas, que era lo unico valioso de LangChain tools.

P: Como escala el sistema si crece el numero de usuarios?

R: El sistema escala horizontalmente con gunicorn (multiples workers) detras de un load balancer. La base de datos PostgreSQL soporta alta concurrencia. Los cuellos de botella conocidos son: (1) el scheduler single-threaded que se resuelve con Celery, (2) el servidor MCP subprocess que se resuelve con SSE, y (3) los hilos de background_tasks que se resuelven con un pool de workers (ThreadPoolExecutor). El Dockerfile incluido facilita el despliegue en Kubernetes o ECS.

P: Por que elegiste WhatsApp y no Telegram o un canal propio?

R: Por adoption rate: WhatsApp tiene el 93% de penetracion en Espana. El publico objetivo (autonomos y PYMEs) ya usa WhatsApp para comunicarse con clientes, por lo que la friccion de adopcion es practicamente nula. Telegram tiene mejor API tecnica pero mucha menor penetracion en el segmento empresarial espanol. Un canal propio requiere desarrollo movil nativo (iOS + Android) o una PWA, lo que esta fuera del alcance del MVP academico.

P: Que aporta el sistema MCP respecto a simplemente llamar las funciones directamente?

R: MCP aporta estandarizacion e interoperabilidad. Con MCP: (1) el LLM descubre las herramientas disponibles en tiempo de ejecucion, no en tiempo de compilacion; (2) el servidor de herramientas puede ser reemplazado sin cambiar el agente; (3) el mismo servidor MCP puede ser consumido por diferentes clientes (Claude Desktop, otros agentes, la propia app); (4) sigue el estandar de la industria, haciendo el sistema interoperable con el ecosistema emergente de MCP servers de terceros.

P: Como manejas el contexto largo en conversaciones extensas?

R: El AgentExecutor recupera las ultimas 10 interacciones del historial (configurable). Esto limita el contexto enviado al LLM a una ventana deslizante, evitando el problema de context window overflow y controlando el costo por token. La limitacion es que el agente 'olvida' conversaciones muy antiguas. Una mejora seria implementar memoria semantica: vectorizar el historial con embeddings y recuperar los mensajes mas relevantes (RAG sobre el historial), no solo los mas recientes.

Documento generado automaticamente para la defensa del TFM.

Ticketia — Plataforma de IA para la Gestion Empresarial Automatizada

2026 | Trabajo Fin de Master